

CS444 A5 Design Document

Code Generation

Arnav Tripathi (a22tripa)

Ryan Maxin (rsmxin)

Andreja Japundzic (ajapundz)

Design and Implementation	3
Overview	3
Organization	3
Code Generation Strategy	4
Runtime Representation	5
Strings, Standard Library, and Native Runtime	5
Trade-Offs and Challenges	6
Testing and Known Issues	6
Testing	6
Known Issues	7
Contributions	7
Thoughts	7
Appendix	8

Design and Implementation

Overview

In the first four assignments, we created a compiler frontend and middlend for scan, parse, build ASTs, resolve names, check types, and reject invalid Joos programs. In A5, we extend this pipeline to include a backend that takes the fully-resolved and type-checked AST and generates 32-bit x86 assembly code.

Organization

The compiler remains organized as a pipeline of distinct passes. The driver runs the frontend and middlend exactly as before, then hands the final typed program to the backend generator. The backend itself is split into several modules.

1. **Generator (/lib/backend/generator.ml):** This is the top level entry point. It takes the compiler's resolved AST's and symbol-table state, and then builds all the backend metadata first, then emits the actual assembly output in stages: one pass for class files, one for the subtype table, one for array files, and one for the startup file `_start`.
2. **Config (/lib/backend/config):** The modules in this folder define the i386 assembly language. Each assembly instruction is represented as a type (`i386.ml`). During code generation, we build lists of these instructions, and then use the defined output function to print the code to output. We also have a module defining registers and conventions (`registers.ml`) as well as a module for outputting unique labels (`label.ml`) (increments and appends counter).
3. **Global Emission (/lib/backend/codegen/global_emission):** The modules in this folder are for emitting assembly files which belong to the whole program, not one specific class. There is a module for outputting the file which contains the selector table (`selectorTableGen.ml`), a module for outputting the file which contains the subtype table (`subtypeTableGen.ml`), a module for outputting individual class information for arrays (`arrayGen.ml`), and a module for outputting the file containing "`_start`" (`startGen.ml`).
4. **Layout (/lib/backend/codegen/layout):** The modules in this folder precompute all the necessary information that later code generation depends on. The pipeline is: assign global ids to runtime types (for tracking table positioning) (`runtimeTypeIndex.ml`); discover every array type so we know which types need an additional array class information file generated (`arrayTypeRegistry.ml`); assign interface selector slots (`selectorSlotIndex.ml`), compute per-class layouts (`classLayoutPass.ml`); compute subtype table (`subtypeTableBuilder.ml`); and return one big lookup structure of all data collected for codegen. The entry point and driver is "`classDataBuilder.ml`" where these steps are detailed.
5. **Per Class Emission (/lib/backend/codegen/per_class_emission):** This folder contains the modules which generate assembly for one AST (class) at a time. The entry point and driver is "`classEmittor.ml`" which takes the basic class layout from "`classLayoutPass.ml`" and generates (in this order) constructor code (`constructorGen.ml`) and method code (`methodGen.ml`). The "`methodGen.ml`" module calls the modules "`stmtGen.ml`" and "`exprGen.ml`" to generate the method bodies. "`exprGen.ml`" has the helper module "`exprGenUtils.ml`".

Code Generation Strategy



Above is a simplified UML diagram of our code generation strategy, the goal here is to show a high level overview of what our code generation strategy was:

1. The entry point is "generator.ml" which passes all the data we collected from the middle end to the "classDataBuilder.ml" which collects all the context needed for code generation.
2. Then, "classEmitter.ml" is called to generate a .s file for each class. This is done by first calling the "classMetadataEmitter.ml" to generate the outline for each class:
0: Pointer to the subtype testing table for this class (4 bytes)
4: Pointer to the selector table for this class (4 bytes)
8: Label for the static field area of this class (4 bytes)
8 + 4 * num_static_fields: Collection of constructor + method labels of this class
8 + 4 * (num_fields + num_labels): Constructor + Method implementations
3. After the outline is generated, "constructorGen.ml" is called to generate all the constructor implementations and then "methodGen.ml" is called to generate all the method implementations. Both of these call the "stmtGen.ml" which calls the "exprGen.ml"
4. Finally, the global files are generated using "selectorTableGen.ml", "subTypeTableGen.ml", "arrayGen.ml", and "startGen.ml."

Additional notes on strategy and implementation:

Method invocation is divided into static and instance calls. Static calls are emitted as direct calls to the target label. Instance calls first evaluate the receiver, check for null, push the receiver and arguments, load the class-data pointer from the object header, follow it to the correct dispatch table, load the appropriate function pointer from the chosen slot, and perform an indirect call. This same overall design also supports interface calls by switching from the class method table to the interface selector table.

Casts and instanceof are implemented using the runtime type-test tables. Instead of recomputing subtype relationships dynamically, the backend simply checks the slot corresponding to the target type. A reference cast raises the exception routine on failure, while instanceof returns 0 or 1. This keeps the generated code relatively small and makes the runtime behavior predictable.

Overall, we followed the suggested implementation in class closely such as using a subtype table and a selector indexed table. Additionally we use **caller pops convention as well as evaluating arguments left to right.**

Backend Conventions

Our backend uses a stack-based calling convention where the caller pushes arguments and also removes them after the call. Local variables are stored in the stack frame at fixed offsets from `ebp`, giving each binding a stable location during code generation.

Objects use a fixed layout: the first word points to class metadata, instance fields are stored at fixed offsets in the object, and static fields are stored separately in the data segment. The class metadata contains pointers to selector tables, subtype-testing data, static storage, and method tables. Interface dispatch uses selector tables, while subtype checks use a precomputed binary matrix.

Labels are generated from class-qualified names and symbol ids to ensure uniqueness, while control-flow labels use fresh counters. The backend uses `eax` for expression results and return values, `ebp` as the frame pointer, and `esp` as the stack pointer. For simplicity, all runtime values are represented as 32-bit words.

Runtime Representation

Our backend uses a uniform runtime object model for classes and arrays. Every heap object begins with one machine word pointing to a runtime descriptor for its dynamic type. For class instances, the remaining words store instance fields in inheritance order, with inherited fields first and subclass fields appended after them. Static fields are stored separately in a class-specific static area rather than inside each object.

Each class runtime descriptor contains the metadata needed for dynamic behavior: a subtype-test table, an interface selector table, a static field area, and a method table. These structures support casts and `instanceof`, interface dispatch, and virtual method calls.

Arrays use a similar representation. An array begins with a pointer to an array runtime descriptor, followed by its length and element storage. Array descriptors are designed to work with the same runtime type system as classes, allowing arrays to participate in subtype checks while still supporting methods inherited from `java.lang.Object`.

Strings, Standard Library, and Native Runtime

Our design keeps the native runtime as small as possible and implements most behavior in Joos itself through the standard library. Strings are the main exception: the backend handles string literals directly by constructing the underlying character array and `String` object at code generation time. Higher-level string operations, however, remain in library code. For example, string concatenation is lowered into ordinary calls to `String.valueOf` and `String.concat` rather than implemented as a special backend primitive. This keeps the backend simpler and allows us to reuse normal method-call generation, while making the correctness of the standard library especially important. The native runtime therefore provides only a small set of low-level services, such as memory allocation, exceptional termination, and a basic output primitive, with most printing and string manipulation logic implemented in the supplied library classes.

Trade-Offs and Challenges

The largest trade-off we made was choosing not to implement any optimization-related content because of time constraints. Instead, we lowered code directly and performed all operations using just two

registers. This made the backend much simpler to build quickly, and it let us take advantage of the metadata infrastructure from earlier assignments right away. The downside, however, is that the generated code will likely have poor runtime performance.

Another important trade-off was deciding to compute runtime layout information globally before code emission. This required more infrastructure up front, since we had to determine field offsets, method slots, and subtype slots ahead of time. In return, though, it made the actual code generation phase much cleaner. Once that layout information was available, the generators could emit code using straightforward lookups instead of recomputing layout details on the fly.

The most difficult part of the assignment was the sheer volume of code needed to complete code generation. Much of the logic was conceptually straightforward, but there was simply a lot of it, and our team only began working on the assignment around a week and a half before the deadline. That time pressure made it harder to build, test, and integrate everything smoothly. One group member, Ryan, also became quite sick during this period, which added further friction to the process.

From a technical perspective, the hardest part was maintaining consistency between the runtime layout and the emitted code. Small mistakes in parameter ordering, object layout, stack cleanup, or field offset calculations would often not appear during compilation, but instead only surfaced later during assembly, linking, or execution. As a result, debugging A5 felt much lower-level and more fragile than in the earlier assignments.

Testing and Known Issues

Testing

For A5 we extended our earlier testing approach into a full compile-link-run pipeline. The script in `test-suite/a5/test_a5.sh` compiles each Joos test, assembles the generated assembly files using NASM, links them using `ld` together with the runtime assembly, and executes the resulting binary. This is important because backend bugs often appear after compilation rather than during compilation.

We also created an additional output regression suite in `test-suite/output` tests. These tests check not only whether a program compiles and runs, but whether it prints the correct output. This suite covers arithmetic, boolean logic, strings, loops, arrays, field access, recursion, null-handling, casts, and `instanceof`. We found these tests especially useful because they exercise larger end-to-end behaviors rather than isolated code generation features.

In addition, we built a complete testing harness spanning all assignments, with a master script that can run the scanner tests, the A1-A5 suites, and the output regression tests in one place. To keep the workflow simple, we used a single `joosc` executable with assignment-specific mode flags such as `--a1` through `--a5`, allowing us to isolate the compiler at the checkpoint relevant to a particular assignment while still sharing one implementation. We also added a `--debug` mode that emits `.dot` snapshots of the AST after each major stage of the pipeline, which can also be rendered as `.png` and `.svg` visualizations. These were especially helpful when debugging frontend and middle-end transformations; examples are included in the appendix at the end.

Known Issues

As of 04/10/2026, there are no known issues with the implementation of our compiler. We have a 100% pass rate on the public test cases provided to us, and we also have included our own output test cases in order to test potential edge case behaviour for the secret tests

Contributions

Andreja

- Built the initial Assignment 5 backend framework and overall codegen pipeline.
- Implemented core expression generation for Joos-to-i386 lowering.
- Designed the foundation of the class metadata/codegen infrastructure.
- Helped stabilize the backend through testing and bug fixing.

Ryan

- Implemented the runtime object model, including object instantiation/allocation and runtime metadata support.
- Implemented selector tables and subtype tables for dispatch and runtime type checks.
- Reorganized the compiler/backend into a cleaner final-project structure and clearer codegen phases.
- Built the master test harness, improved driver/debug workflow, and cleaned up shared backend/codegen infrastructure.

Arnav

- Implemented the `_start` entry point and static field initialization logic.
- Added type-checking changes to record resolved method/constructor targets for reuse in codegen.
- Refactored array runtime generation and helped resolve interface dispatch/access bugs.
- Contributed to final documentation, including the initial Assignment 5 report draft.

Thoughts

Assignment 5 was the most concrete stage of the compiler so far because it forced us to connect the semantics of Joos to an actual runtime model. Earlier assignments mainly asked whether a program was valid. This assignment asked us to define how valid programs should execute, which is a much lower-level and less forgiving task.

OCaml was especially helpful during A5 because algebraic data types and pattern matching fit code generation very naturally. They were especially helpful in AST traversals and in defining the small internal assembly language. At the same time, debugging backend issues in OCaml was less pleasant than debugging higher-level semantic passes, because many A5 problems are ultimately about concrete machine behavior, stack state, and memory layout rather than type structure.

The main lesson we took from A5 is that code generation is not just about writing assembly instructions. It is really about designing a consistent runtime representation for objects, arrays, methods, and types, and then ensuring every part of the compiler agrees on that representation. That was the hardest part of the assignment, but also the part that taught us the most.

Appendix

```
1  // PARSE_WEEEDER
2  public class J1_IntInit {
3
4      public J1_IntInit(){
5
6      public static int test() {
7          int x = 123456;
8          return x - 123333;
9      }
10 }
```

Figure 1: A basic Joos program

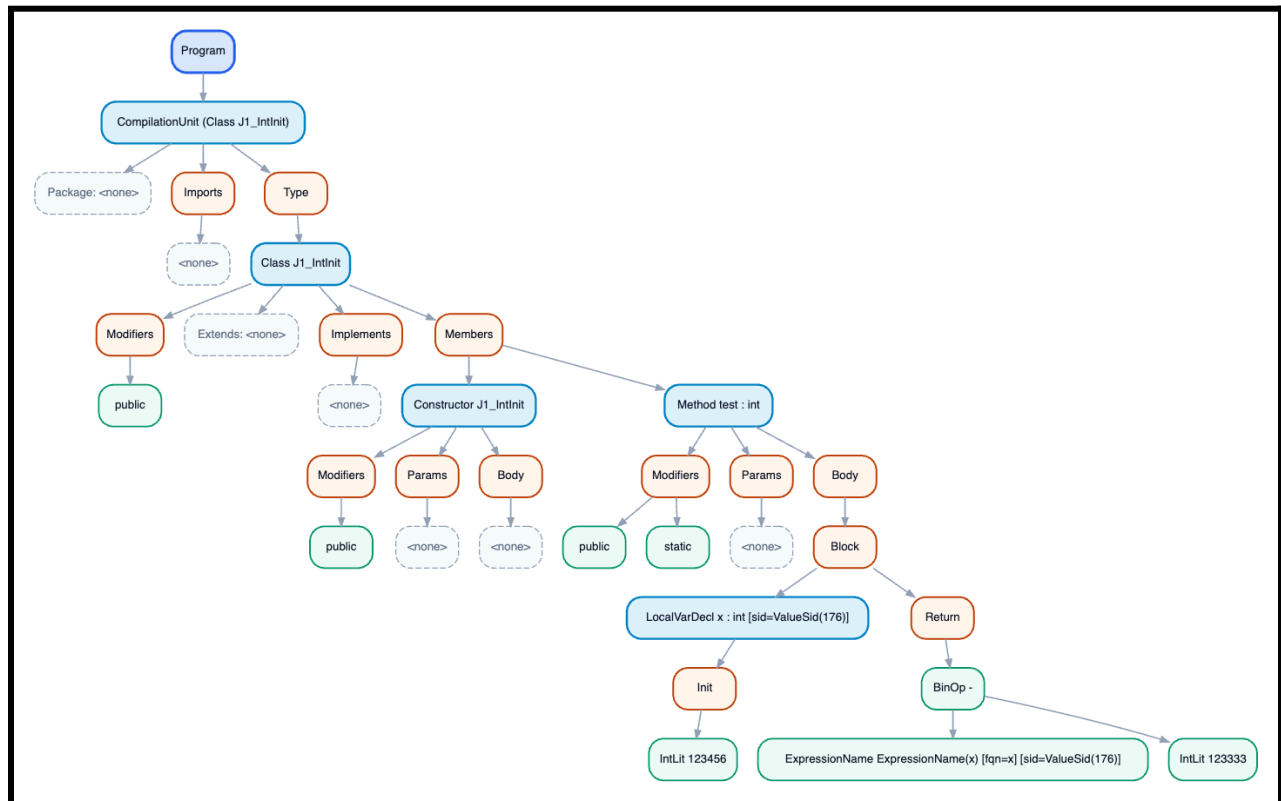


Figure 2: The type checked AST associated with figure 1, visualized

Final Report Card					
Suite	Status	Total	Passed	Failed	Time
Scanner	PASS	49	49	0	00:01
A1	PASS	325	325	0	00:09
A2	PASS	228	228	0	00:31
A3	PASS	323	323	0	00:42
A4	PASS	67	67	0	00:08
A5	PASS	117	117	0	00:59
Output Tests	PASS	100	100	0	00:49
Suites	7/7 passed				
Tests	1209/1209 passed				
Overall	PASS				

Figure 3: The output of our complete testing harness